DRIVER MATCHING — CURRENT SYSTEM

How Driver Matching Works Today, and What to Fix

Our understanding of the current engine — the exact numbers, the algorithm, the problems, and how to fix them

Prepared by: Akshay · **Based on:** goezz-nodejs-server (matching engine) · **Reference:** Driver matching (UAD)

Summary of our understanding. This document describes exactly what the current matching engine does, with the real numbers and conditions from the code, then lists the problems we are facing today and how to fix each one. The key finding: most of the system already exists and works. The main issue is that several useful scoring rules are switched off in the code, which is why a family's regular driver is sometimes not matched.

1 When matching runs (triggers)

A ride group with no driver is pushed onto a queue (AWS SQS), and a Lambda runs the match engine. It is triggered by:

- A **daily job at 7:00 PM Pacific** that publishes all unclaimed ride groups roughly 6 days ahead.
- An admin clicking **Publish match** for a single ride group or a date range.
- A **new request** being created, or a ride being **edited or cancelled** (re-queued).
- A driver **declining or abandoning** a claimed ride (re-queued to find a replacement).
- A **retry loop** (AWS Step Function) that re-runs the match every 30 minutes while the ride stays unclaimed.

Only unclaimed ride groups that have their stops saved and are not cancelled are queued.

2 The exact numbers used today

These are the current values in the code, all editable from the admin **Meta Data** screen (table `admin_meta_data`):

Setting (config key)	Current value
Starting search radius `driver-proximity-miles`	**5 miles** (code falls back to 4 if the value is missing)
Radius increase per expansion `...-match-increment`	**5 miles**
Maximum expansion steps `max-expansion-attempts`	**5** → radius grows 5 → 10 → 15 → 20 → 25 → 30 miles
Drivers matched per attempt `number-drivers-to-match`	**5**
Maximum match attempts `max-match-attempts`	**5**, then the ride is marked FINAL and matching stops
Wait between retry attempts `match-interval`	**1800 seconds (30 minutes)**
Last-minute threshold `last-minute-threshold-minutes`	**1440 minutes (24 hours)** before ride start
Travel-time buffer `feasible-travel-buffer-minutes`	**10 minutes**
Prior / next ride qualifying window	**40 minutes** each side
Gender preference `gender-preference-enforced`	**Enforced (1)** — used as a hard filter, not points

> **To verify in code:** an earlier ticket mentioned 4 miles initial / 2-mile steps / 12 mile max. The engine code shows 5 / +5 / 30. The live values in `admin_meta_data` are the source of truth — confirm the actual DB values before quoting these in a meeting.

3 The algorithm — how a driver is selected

For each ride group, the engine runs these steps in order:

A **Exclusion list.** Remove drivers who must never be offered this ride: already offered it, offered before (history), declined or abandoned it, or blocked by the parent.

B **Hard filters (must pass all).** Keep only drivers who are onboarding **passed** and **active**, have an **active primary vehicle** with **enough seats** (seats ≥ max kids in the car at once), match the **gender preference** when the rider requires it, have **no other claimed ride overlapping in time**, and **can drive from their previous ride to this one, and to their next ride, in time**. This travel-feasibility check uses **Google Routes API with live traffic** plus a 10-minute buffer.

C **Proximity check (within the radius).** A driver passes if **any one** of these is true: their **service location** is within the radius of the pickup (first stop), OR their **previous ride** ends within the radius of the pickup, OR their **next ride** starts within the radius of the drop-off. Distance here is **straight-line** (geolib), not driving distance.

D **Expansion loop.** If fewer than 5 qualified drivers are found, the radius grows by 5 miles and the search repeats — up to 5 times (to 30 miles) — adding only as many new drivers as still needed.

E **Scoring.** Each qualified driver is given points (Section 4).

F **Sort, shuffle ties, take top 5.** Sort by total points (highest first), randomly shuffle drivers with equal scores for fairness, take the top 5, insert them as offers, and push a notification/socket to each driver's app. If the ride is still unclaimed after 30 minutes, the retry loop runs again.

Bundled ride groups: only the earliest ride (the "anchor") runs matching; matched drivers are copied to the other rides in the bundle. Hard filters are checked against every ride in the bundle; proximity only against the anchor.

4 How drivers are scored today

Every qualified driver starts at 0 points. Only these scoring rules are **active** in the code right now:

Active scoring rule	What it does
Completed rides	Points based on rides completed in the last 365 days, using bands (e.g. 10 rides = 10 pts, 30 rides = 30 pts — takes the highest band whose value ≤ count).
Gender preference	Only applies when gender is **not** enforced as a filter — then a matching gender adds points (default 125).
Reliability penalty	Subtracts points for active penalties (quick abandon, abandon, last-minute abandon).

Highest total wins; ties broken by random shuffle. **Scoring rules that exist in the code but are switched off (commented out):** route/location history, rider history, and distance-based points — see Section 5.

5 Problems we are facing, and how to fix them

These are the real issues in the current code, each with its fix.

PROBLEM 1 — THE REGULAR DRIVER IS NOT PRIORITISED

The scoring rules that reward a driver for **having served this route before** (`locationMatch`) or **this rider before** (`riderMatch`) exist in the code but are **commented out**. So a family's regular driver scores the same as any other nearby driver. This is why drivers like Karen and Shahin, who normally do the route, are sometimes not matched.

FIX

Enable the existing `locationMatch` and `riderMatch` scoring and give them a high weight, so the regular driver rises to the top. Low risk — the logic is already written, it just needs turning on and tuning.

PROBLEM 2 — BEING CLOSER GIVES NO ADVANTAGE

The **distance scoring** (`distanceMatch`) is also commented out. Today proximity is only a pass/fail filter, so a driver 25 miles away can be chosen over one 3 miles away if their other scores are equal.

FIX

Enable `distanceMatch` so closer drivers rank higher within the qualified set, alongside the route/rider history rules above.

PROBLEM 3 — "RECENTLY COMPLETED RIDES" ALWAYS SCORES ZERO

A rule meant to reward drivers active in the last 7 days exists, but its config key (`rides-recently-completed-points`) is **never read** from `admin_meta_data` , so it always gives 0 points even when configured.

FIX

Add that key to the list the engine reads, so recent activity counts (if we want it to).

PROBLEM 4 — SERVICE ADDRESS MAY BE MISSING FOR SOME DRIVERS

Matching relies on the driver's **service location**. A driver with no service address on file cannot be matched. Service address is not currently guaranteed at onboarding.

FIX

Make service address a **required field at onboarding**, run a DB query to find existing drivers with no service address and flag them, and add reminders (e.g. weekly) for drivers to keep it current.

PROBLEM 5 — LIVE LOCATION IS NOT USED AT ALL

The engine matches on the driver's fixed **service location** and neighbouring rides — it does **not** use the driver's live location. The team wants live location used as an extra check, when available at least 2 hours before the ride.

FIX

Add live location as a **new optional condition** on top of service address, used only when a recent location is available in the 2-hour window. Service address stays the primary filter; home address is not used.

PROBLEM 6 — A DRIVER WITH NO PRIMARY CONTACT IS SILENTLY DROPPED

If a driver has no primary contact record (`user_comm FK_COMMID = 1`), the database join removes them **with no log**, so they never appear even in the "why not matched" list — hard to debug.

FIX

Handle this case so such drivers are either included or at least logged as disqualified with a clear reason.

PROBLEM 7 — SMALLER CODE ISSUES

The driver counts for last-minute and final runs are read but ignored (always 5). The radius default differs in two places (5 vs 4). The proximity check only uses the pickup stop, not the drop-off. The admin "Match Engine" test page does not work (the backend returns nothing).

FIX

Clean these up alongside the main work: use the intended counts, align the default radius, decide on the drop-off check, and fix or remove the test page.

6 The good news

Most of what the team wants **already exists**: the radius search with expansion, the attempt and retry loop, service-address-based proximity, real driving-time feasibility (Google Routes with traffic), and a full debug log of why each driver was or was not matched. The main work is to **turn on scoring rules that are switched off** (route history, rider history, distance) — which directly fixes the regular-driver problem — and to **add the two new pieces** the team asked for (the live-location check and mandatory service address). This is far less work and far less risk than rebuilding matching from scratch.

Before presenting as final: the headline finding (route/rider/distance scoring is commented out) should be confirmed directly in `points/calculateDriversMatchPoints.ts`, and the live radius/attempt values confirmed in `admin_meta_data`, since both are the load-bearing claims in this document.

Prepared by Akshay, based on a review of the current matching engine code. Values reflect the current defaults in the admin Meta Data configuration and may be tuned. Exact weights and thresholds to be confirmed with the team.